

Diff-aware deep learning for intent classification and implementation-hole detection in RTL code changes

[Full Name 1], [Full Name 2]

Deep Learning Course Project | NLP / Code Understanding / Hardware Design Automation | PyTorch

Manuscript draft prepared for the course final report; replace placeholders before submission.

Abstract

Modern hardware repositories contain thousands of small RTL changes whose intent and completeness are difficult to review at scale. We study whether deep learning models can classify the intent of SystemVerilog code changes and detect synthetic implementation holes by comparing two versions of the same file. From the OpenTitan commit history, we mined 5,903 raw file-level RTL candidates, retained 2,843 weakly labelled real examples, and constructed 5,500 final examples after paired synthetic-hole generation. The proposed model is a PyTorch hierarchical Transformer that encodes unified diffs as typed lines, uses a bidirectional line encoder and line-type embeddings, and shares a representation between an intent head and a hole-detection head. Across three seeds, TF-IDF with logistic regression is the strongest intent classifier (macro-F1 = 0.4197), revealing substantial weak-label leakage from commit-message-derived labels. In contrast, the hierarchical Transformer is best for implementation-hole detection (F1 = 0.6029, AUROC = 0.7352), outperforming flat neural baselines and TF-IDF. Ablations show that line-type embeddings are critical for hole detection, reducing F1 from 0.6029 to 0.4724 when removed. These results support the value of diff-aware structure for completeness analysis while clarifying the limits of keyword-derived intent supervision.

Significance

- **Problem relevance.** Hardware RTL code review requires reasoning about small changes to guards, resets, assertions, parameters and state-machine logic.
- **Main technical contribution.** A line-type-aware hierarchical Transformer represents diffs at token, line and file levels and jointly optimizes intent and completeness tasks.
- **Key empirical result.** The proposed architecture substantially improves hole detection over flat neural models, while the intent task is exposed as weak-label limited.
- **Course alignment.** The study includes a PyTorch implementation, explicit baselines, systematic metrics, ablations, error analysis and reproducibility instructions.

Introduction

Code-change understanding has become an important setting for deep learning on source code because a change is not merely a final program snapshot: it contains a structured comparison between the previous and new versions of a program. In hardware design, this structure is especially important. A single missing reset assignment or dropped guard can cause behavioural divergence even when the diff remains syntactically plausible. This project therefore studies code changes in SystemVerilog RTL, focusing on two supervised tasks: classifying the intent of a change and detecting whether a diff may contain an implementation hole.

Prior work motivates modelling the patch itself. CC2Vec learns distributed representations of code changes guided by log messages; PatchNet uses a hierarchical representation that mirrors the structure of patches; DeepJIT predicts defect-prone commits from code changes; and CodeReviewer uses pre-training objectives tailored to code-diff and review tasks. The present study adapts these ideas to open-source RTL by preserving the semantics of added, deleted and context lines in unified diffs and evaluating whether those line types improve downstream performance.

The research question is deliberately narrow and testable: does a diff-aware hierarchical neural model provide measurable value over simpler lexical and flat sequence baselines when predicting change intent and identifying incomplete implementation changes in RTL files? The answer is mixed but informative. The neural architecture helps the completeness task, whereas the intent task is dominated by label-generation artefacts and lexical baselines.

Results

Dataset construction produced a compact RTL change benchmark

The data source was the OpenTitan repository. The mining policy kept commits touching `hw/**/rtl/*.sv`, excluded test, generated, vendor, third-party and build paths, removed merge and revert commits, and restricted examples to manageable file-level diffs.

From 5,903 raw RTL candidates, the pipeline retained 2,843 real examples with unambiguous weak intent labels. Paired synthetic holes were then generated inside changed regions, producing 5,500 final examples grouped into 2,843 real/mutated pairs. Chronological splitting with pair preservation yielded 3,873 training examples, 830 validation examples and 797 test examples.

Intent labels are moderately imbalanced: feature_or_behavior_change accounts for 40.7% of examples, bug_fix for 33.8%, configuration_timing for 17.0%, and refactor_cleanup for 8.5%. Hole labels are nearly balanced, with complete examples at 51.7% and synthetic_hole examples at 48.3%. Synthetic holes are not uniformly distributed: drop_added_line dominates the full dataset, with rarer operator flips, missing reset/default assignments and hunk-level mutations.

Dataset Overview

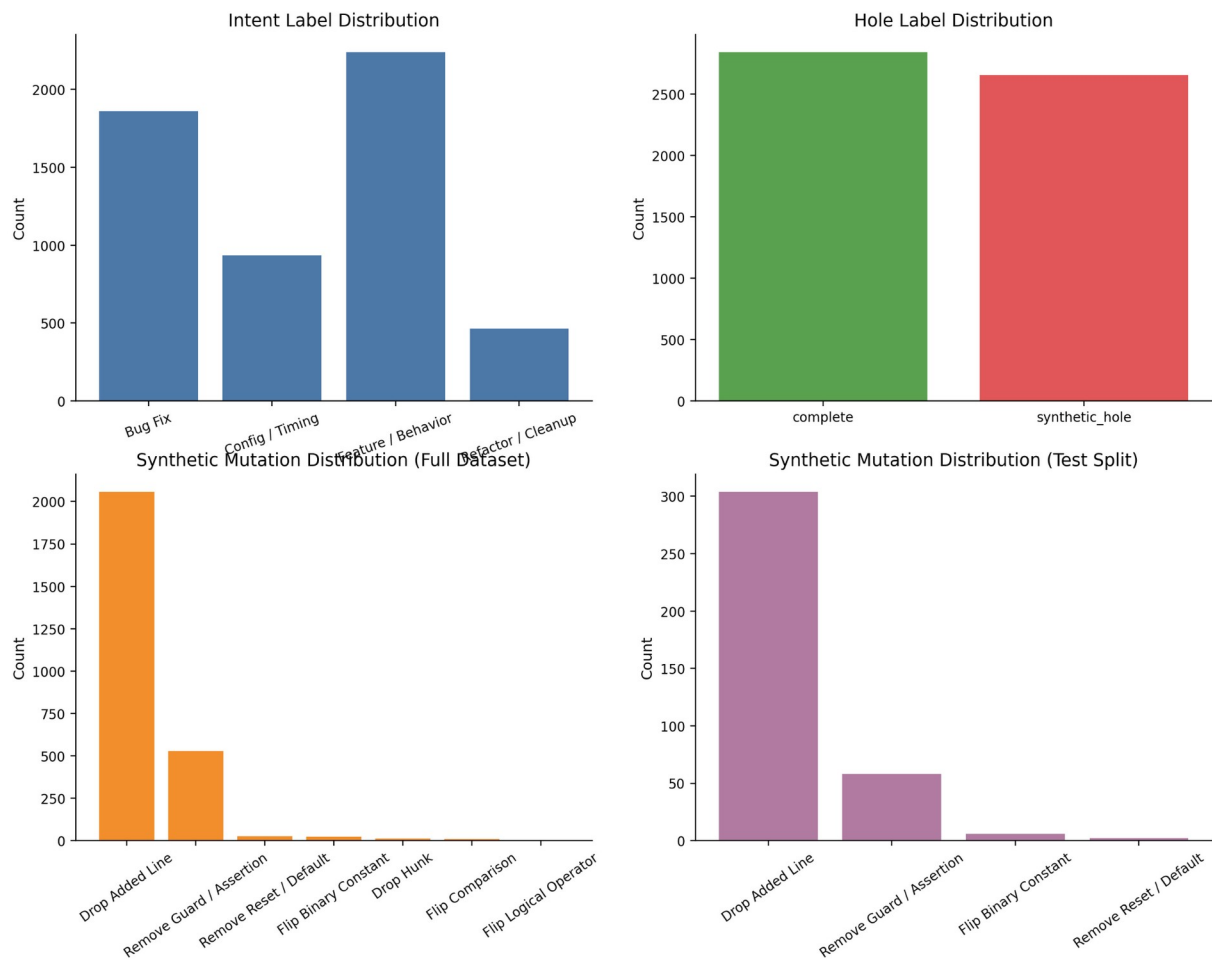


Figure 1. Dataset overview. The final processed benchmark contains moderately imbalanced intent classes, nearly balanced hole labels, and a synthetic-hole distribution dominated by dropped added lines. The test split has the same qualitative imbalance in mutation types.

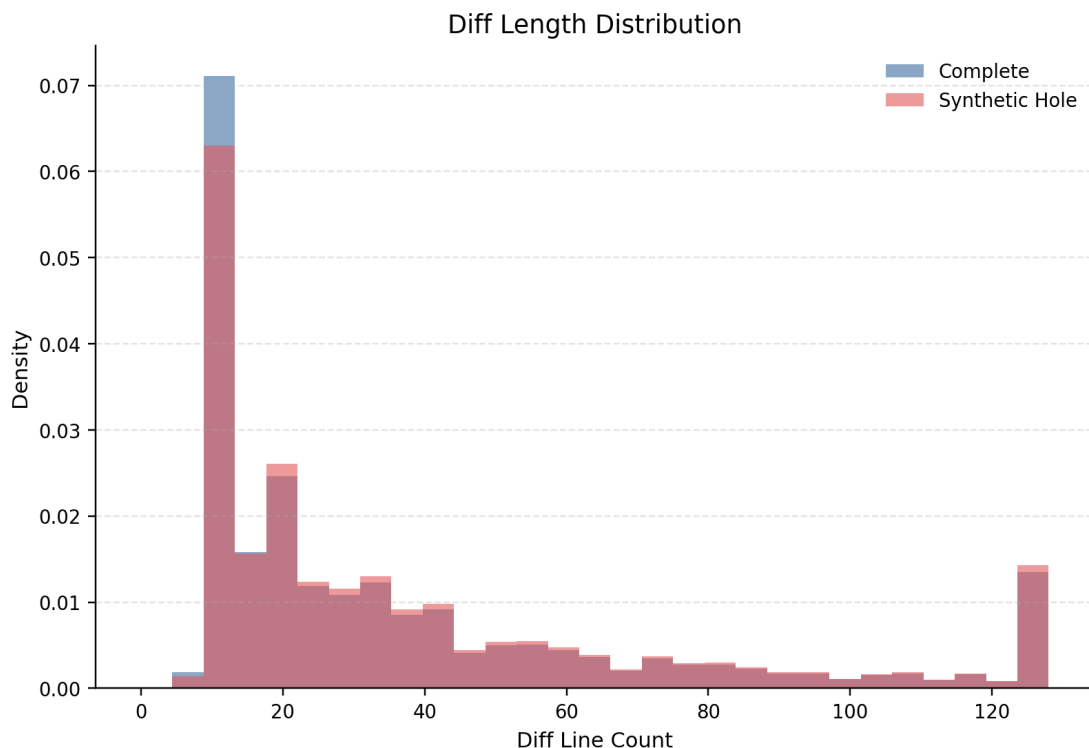


Figure 2. Diff length distribution. Complete and synthetic-hole examples occupy a similar range of diff lengths, indicating that the hole-detection task cannot be solved by length alone.

A line-type-aware hierarchical Transformer was compared with lexical and flat neural baselines

Four primary systems were evaluated. TF-IDF with logistic regression provided a strong lexical baseline using word and character n-grams. A bag-of-tokens MLP tested whether learned embeddings plus shallow pooling were sufficient. A flat BiGRU tested sequential modelling over tokenized diffs. The proposed hierarchical Transformer tokenizes each diff line, encodes tokens into line vectors using a bidirectional GRU, adds learned line-type embeddings for ADD, DEL, CTX, HUNK, FILE and PAD, applies a 2-layer Transformer encoder over line representations, and predicts intent and hole labels through two output heads.

Model	Runs	Intent acc.	Intent macro-F1	Hole acc.	Hole F1	Hole AUROC
TF-IDF + LR	3	0.5132	0.4197	0.5734	0.5430	0.6123
Bag-of-tokens MLP	3	0.4049	0.2961	0.5529	0.3293	0.5488
Flat BiGRU	3	0.3998	0.3023	0.5822	0.4204	0.5942
Hierarchical Transformer	3	0.4065	0.3007	0.6725	0.6029	0.7352

Table 1. Main test results averaged over three seeds. TF-IDF is strongest for intent classification, whereas the hierarchical Transformer is strongest for hole F1 and AUROC.

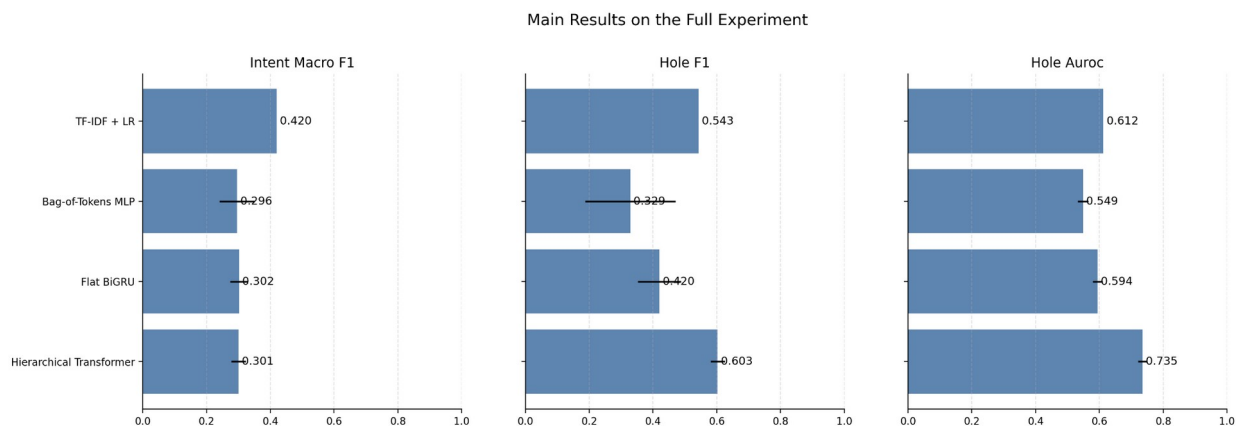


Figure 3. Main full-experiment results. TF-IDF dominates intent macro-F1, but the hierarchical Transformer achieves the best hole F1 and hole AUROC.

Intent classification is limited by weak-label leakage

The strongest intent classifier is not a deep neural model but TF-IDF with logistic regression, which obtains an intent macro-F1 of 0.4197. The hierarchical Transformer reaches only 0.3007 macro-F1. Per-class analysis shows that `feature_or_behavior_change` is the easiest class, whereas `refactor_cleanup` is rare, semantically ambiguous and frequently confused with bug fixes or behaviour changes. The message-only TF-IDF control reaches 0.9855 macro-F1, demonstrating that the weak label is almost recoverable from the same commit-message channel used to create it. Thus, the intent task is useful as an engineering exercise but is not yet a clean evaluation of pure code-diff understanding.

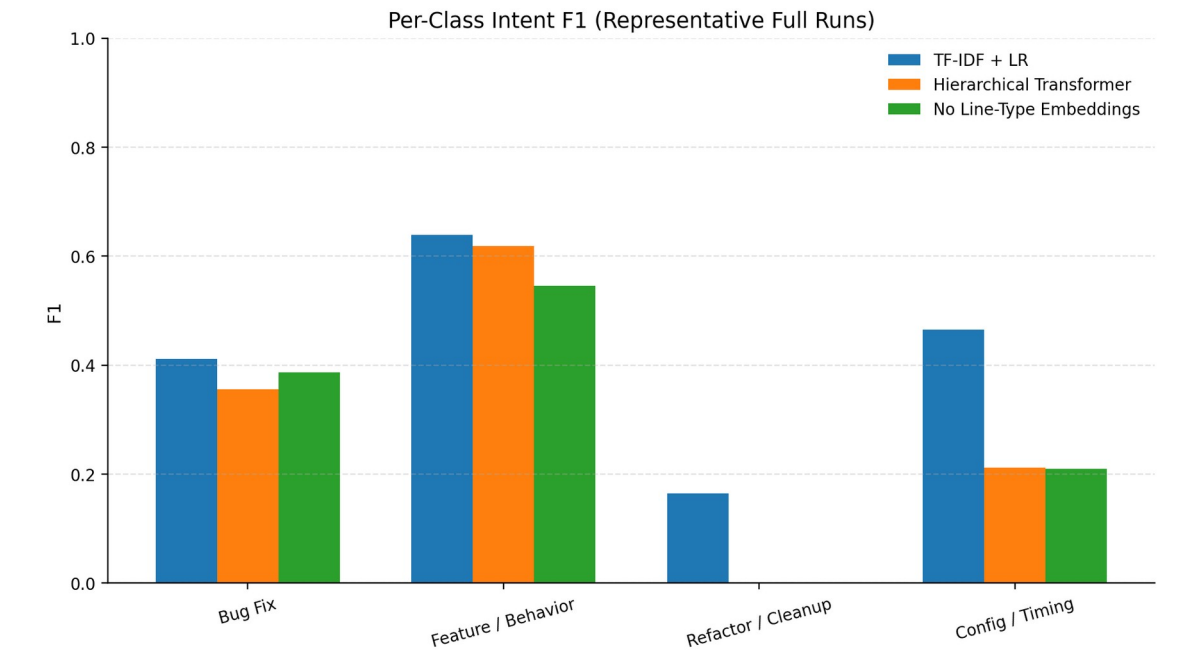


Figure 4. Per-class intent F1. The TF-IDF baseline is strongest overall, while all models struggle with the rare refactor/cleanup class.

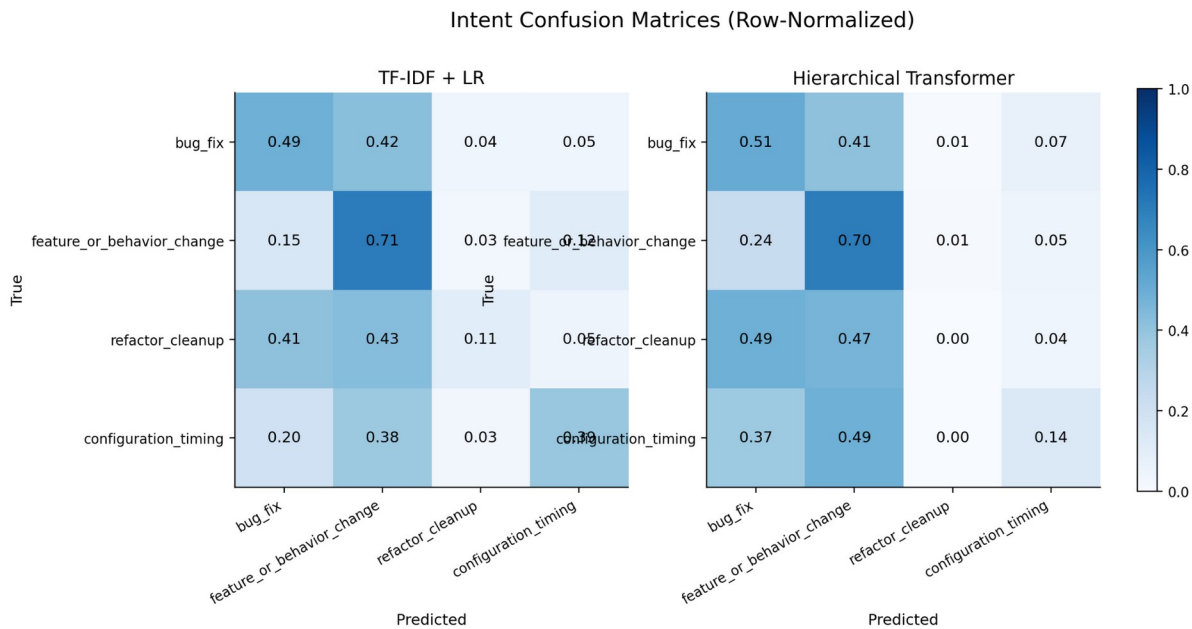


Figure 5. Intent confusion matrices. The hierarchical Transformer overpredicts `feature_or_behavior_change` and fails to recover `refactor_cleanup` in the representative run, consistent with class imbalance and label noise.

Implementation-hole detection benefits from diff-aware structure

Hole detection tells a different story. The hierarchical Transformer achieves hole F1 of 0.6029 and AUROC of 0.7352, outperforming TF-IDF (F1 0.5430, AUROC 0.6123), bag-of-tokens MLP (F1 0.3293, AUROC 0.5488) and flat BiGRU (F1 0.4204, AUROC 0.5942). Representative confusion matrices explain the gain: TF-IDF and the hierarchical model find almost the same number of synthetic holes, but the hierarchical model substantially reduces false positives on complete diffs. This suggests that the model learns a more conservative representation of what an incomplete change looks like, rather than merely responding to suspicious tokens.

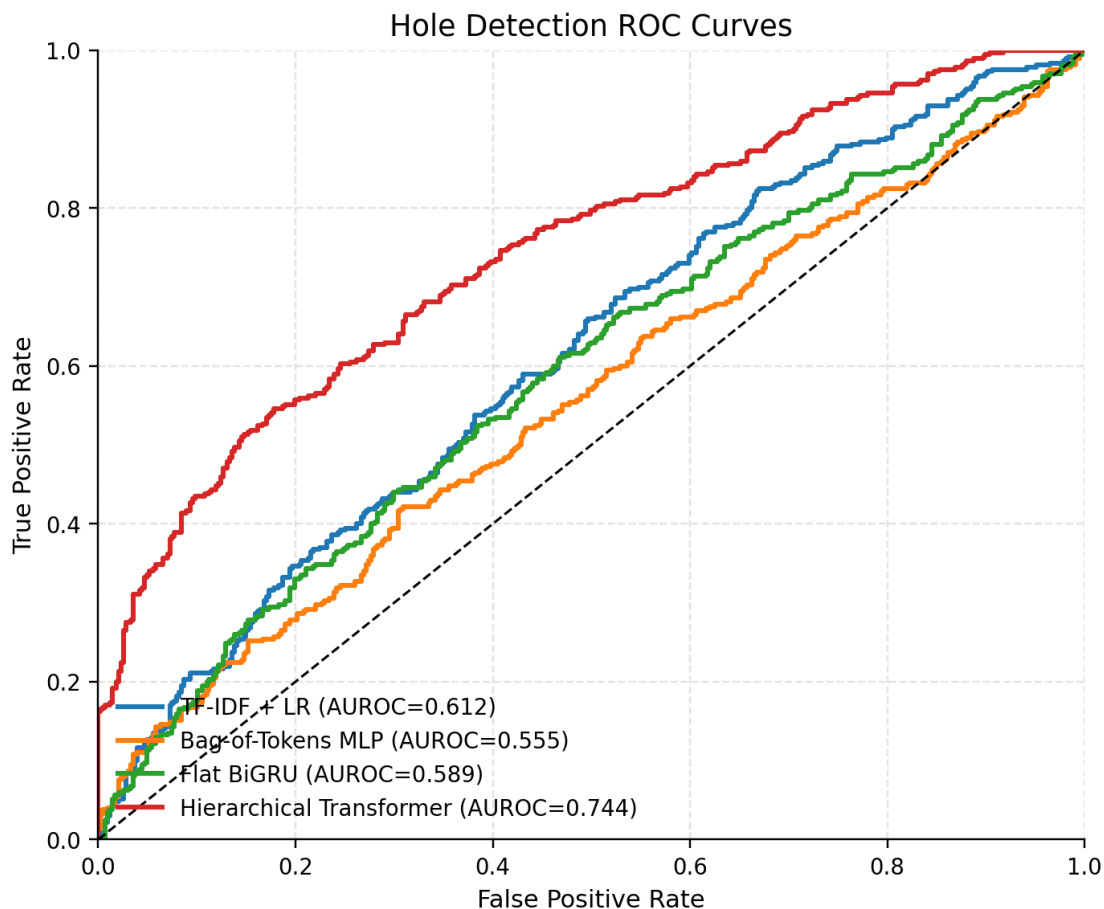


Figure 6. Hole-detection ROC curves. The hierarchical Transformer yields the largest representative AUROC, indicating improved ranking of complete versus synthetic-hole examples.

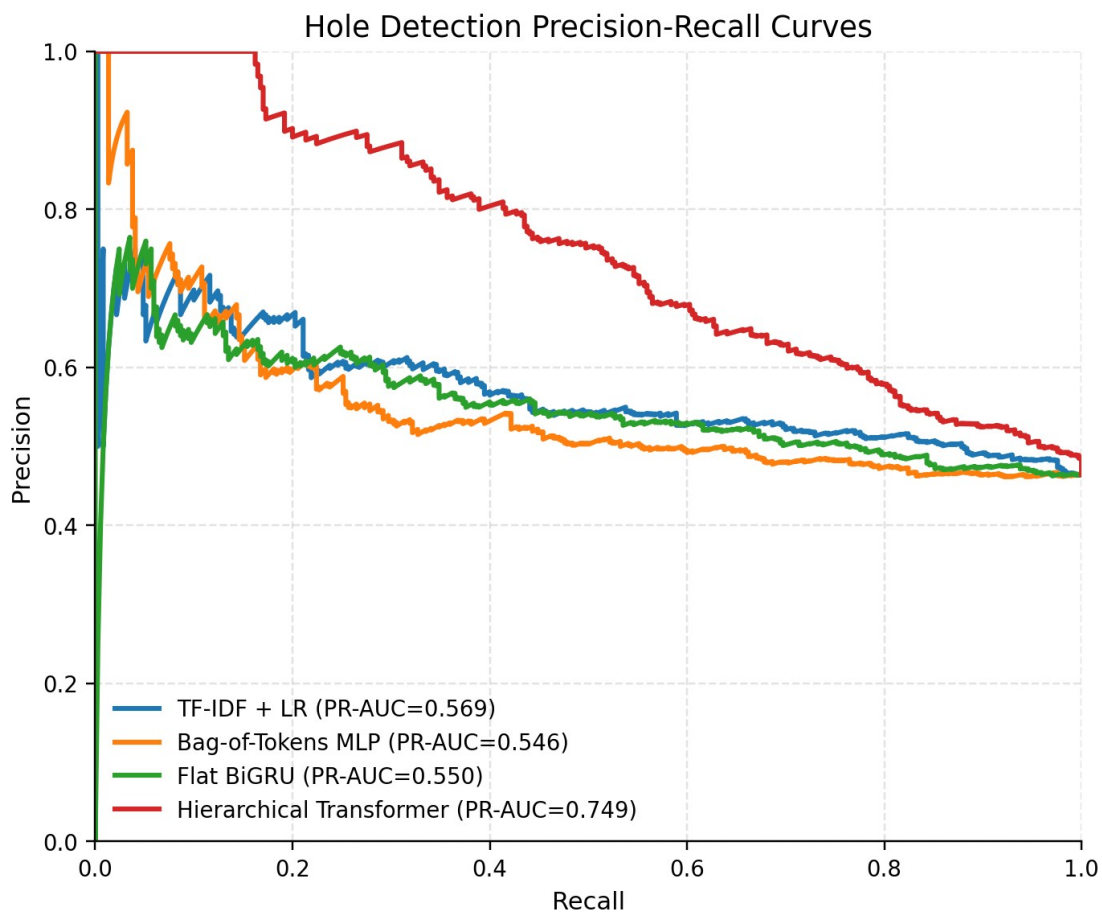


Figure 7. Hole-detection precision-recall curves. The proposed model maintains higher precision over a broad recall range than the flat baselines.

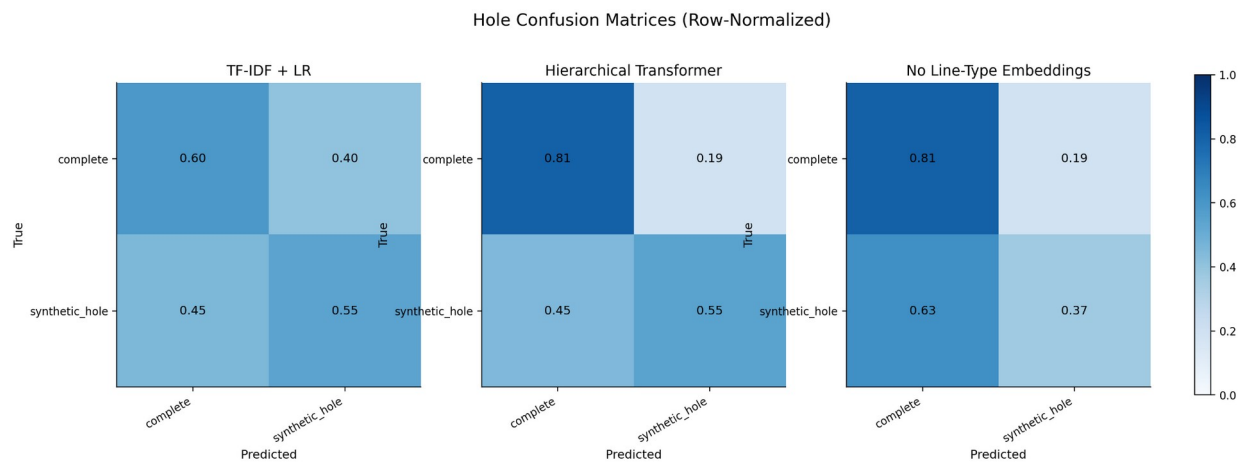


Figure 8. Row-normalized hole confusion matrices. Compared with TF-IDF, the hierarchical model reduces false alarms on complete examples; removing line-type embeddings primarily hurts synthetic-hole recall.

Ablations isolate the contribution of line-type embeddings

The primary architectural ablation removes line-type embeddings from the hierarchical model while preserving the remaining architecture. Across three seeds, this reduces hole F1 from 0.6029 to 0.4724 and AUROC from 0.7352 to 0.6294, while leaving intent macro-F1 essentially unchanged. This is the strongest evidence for the central hypothesis: added, deleted and context lines should not be treated as interchangeable tokens. In the representative confusion matrix, removing line-type embeddings leaves true negatives and false positives unchanged but loses 66 true positives on synthetic holes, showing that line-type information helps the model recognize missing or suspicious changes.

Ablation	Runs	Intent macro-F1	Hole acc.	Hole F1	Hole AUROC
No context lines	1	0.2677	0.6600	0.5937	0.7126
No line-type embeddings	3	0.3028	0.5893	0.4724	0.6294
Hole-only training	1	0.1813	0.6587	0.6354	0.7163
Intent-only training	1	0.3193	0.4743	0.5797	0.4947
Message-only TF-IDF	1	0.9855	0.5295	0.4361	0.5450

Table 2. Ablation results. The no-line-type ablation is the central multi-seed ablation and shows a large degradation in hole detection.

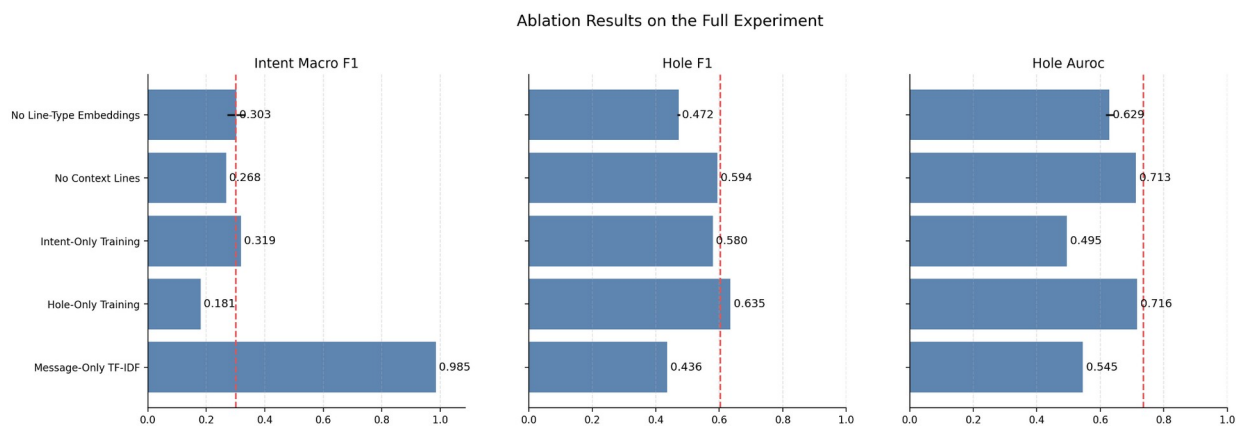


Figure 9. Ablation results. Dashed red lines denote the full hierarchical Transformer baseline; line-type embeddings are especially important for hole F1 and AUROC.

Mutation-type and pairwise analyses reveal what the detector learns

The detector is strongest on visible missing-line patterns and weaker on subtle semantic edits. In the test split, `drop_added_line` accounts for 304 of 370 synthetic holes; the hierarchical model recalls 60.9% of these examples. By contrast, guard or assertion removal reaches 29.3% recall, `flip_binary_constant` has only six test examples and zero recall, and `reset/default` removal has only two test examples. Pairwise ranking analysis further shows that TF-IDF almost perfectly ranks synthetic examples above their paired real examples, while the hierarchical model shows a larger average hole-score margin. These results indicate that ranking and thresholded classification capture different behaviours: lexical features may rank paired mutations well, while the hierarchical classifier provides better calibrated binary detection under the selected threshold.

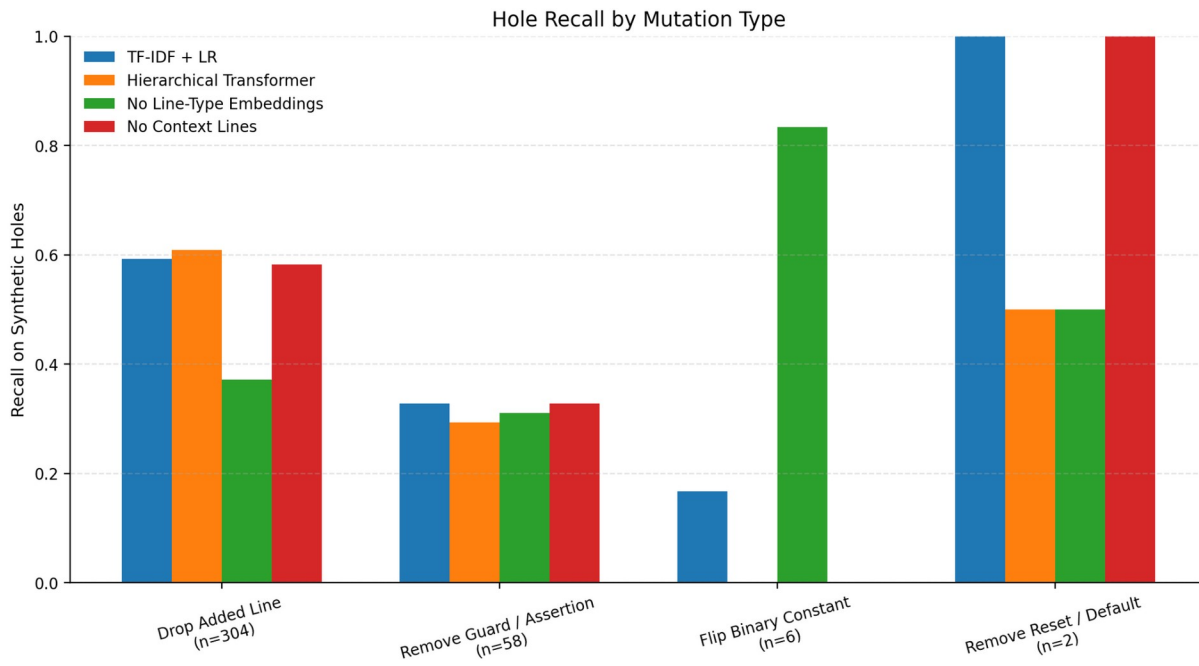


Figure 10. Hole recall by mutation type. The test set is dominated by dropped added lines, while rare semantic mutation types remain difficult to evaluate robustly.

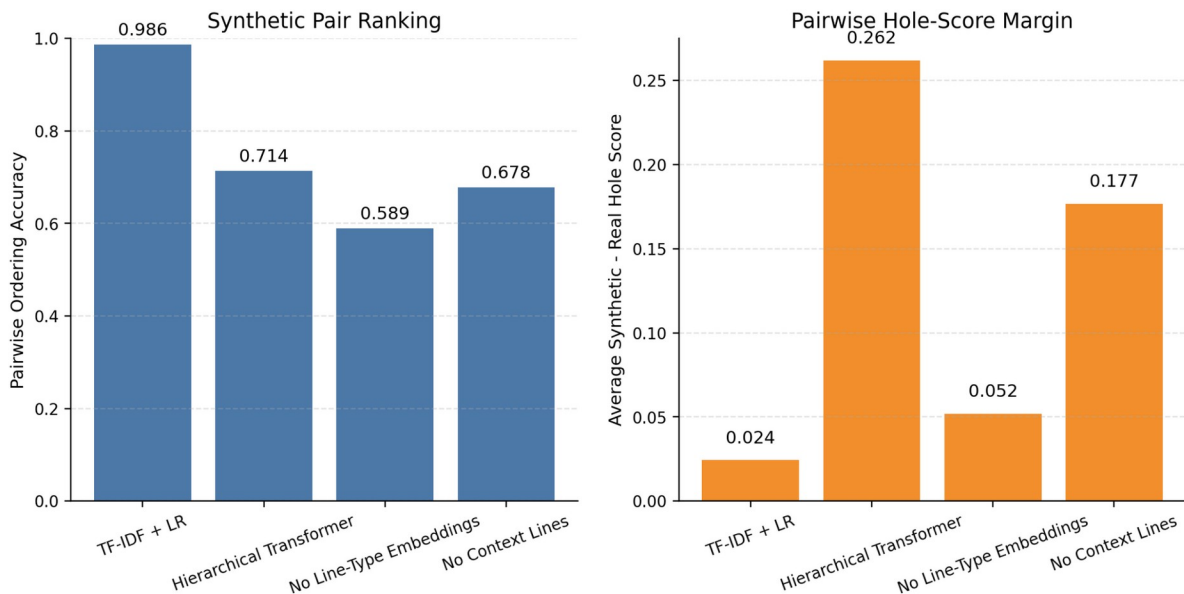


Figure 11. Pairwise synthetic-hole ranking. TF-IDF ranks paired synthetic holes above real examples most consistently; the hierarchical model produces a larger average hole-score margin.

Discussion

The project produces a scientifically useful negative-and-positive result rather than a uniformly positive deep-learning story. For intent classification, the weak labels are too tightly tied to commit-message keywords, and a lexical baseline is the most appropriate model. This outcome is important because it prevents overstating neural progress on an artefactual benchmark. The message-only ablation should be treated as a validity check: if commit messages generate the labels, then commit-message text must not be allowed to define a claim about code understanding without manual relabelling or a leakage-resistant protocol.

For hole detection, the evidence supports the central architectural hypothesis. Complete and synthetic-hole examples have similar length distributions, and the proposed model improves both hole F1 and AUROC over flat neural baselines. The line-type ablation is particularly persuasive because it changes one targeted component and produces a large degradation in the target

metric. This suggests that the semantics of a diff are not just in the tokens but also in whether a token appears in an added line, a deleted line or surrounding context.

The main limitations are also clear. Synthetic holes are dominated by dropped added lines, so the benchmark underrepresents subtle hole types such as comparison flips, logical-operator flips and missing resets. Several ablations were run on a single seed because of compute constraints. The intent class refactor_cleanup is rare and noisy. Finally, the model reads textual diffs rather than synthesizable hardware structure; future work could incorporate ASTs, elaborated netlists, signal-use graphs or formal linting outputs.

Overall, the best-supported conclusion is that diff-aware deep learning is valuable for suspicious-change detection in RTL, but a reliable intent benchmark requires stronger supervision. A publishable next step would be to manually label a stratified subset, rebalance mutation operators, and test whether line-type-aware encoders remain beneficial under cleaner labels and more semantically diverse holes.

Methods

Data source and filtering

OpenTitan was cloned locally, and commits were mined chronologically. The pipeline retained SystemVerilog RTL changes under `hw/**/rtl/*.sv` and excluded paths containing `/dv/`, `/test/`, `/tests/`, `/vendor/`, `/third_party/`, `/generated/`, `/build/` and `/out/`. Merge commits, pure reverts, empty messages and overly large diffs were removed. The unit of analysis was a file-level unified diff, with a preference for one RTL file per commit and a fallback cap of three RTL files per commit.

Weak intent labels

Intent labels were derived from mutually exclusive commit-message keyword groups: `bug_fix`; `feature_or_behavior_change`; `refactor_cleanup`; and `configuration_timing`. Examples with no keyword match were discarded, and examples with multiple maximal label groups were treated as ambiguous and removed. This conservative filtering improves label precision at the cost of coverage, but it also creates the leakage issue analysed in Results.

Synthetic-hole generation

Each accepted real example was labelled complete. A paired synthetic_hole example was then generated by applying controlled mutations within the changed region: removing an added guard or assertion, removing a reset/default assignment, dropping one added line, dropping an added hunk, flipping comparison operators, flipping logical operators or flipping simple binary constants. Original/mutated pairs were kept together during chronological splitting so that no pair crossed train, validation and test boundaries.

Preprocessing and tokenization

Unified diffs were normalized into typed lines, including FILE, HUNK, CTX, DEL and ADD. A SystemVerilog-aware tokenizer preserved identifiers, keywords, width literals, operators, punctuation and macros. Vocabulary construction used the training set only. The executed configuration used `max_lines_per_diff = 128`, `max_tokens_per_line = 64` and `max_total_tokens_flat = 2048`; truncation prioritised added and deleted lines and then nearby context.

Model architecture

The proposed PyTorch model uses token embeddings and token-position embeddings, a bidirectional GRU line encoder, optional line-type embeddings, a 2-layer Transformer encoder over line representations, attention pooling, and two output heads. The intent head predicts four classes, and the hole head predicts complete versus synthetic_hole. Baselines include TF-IDF with logistic regression, a bag-of-tokens MLP and a flat BiGRU with attention pooling.

Optimization and model selection

Neural models were trained with AdamW and cross-entropy loss. The multitask objective was $L = L_{\text{intent}} + L_{\text{hole}}$. For single-task ablations, only the corresponding head loss was optimized. Validation checkpoint selection used intent macro-F1 for intent-only runs, hole F1 for hole-only runs, and intent macro-F1 plus hole F1 for multitask runs. Main runs used seeds 13, 42 and 123; auxiliary ablations were reference-seed analyses unless otherwise noted.

Evaluation

Intent metrics include accuracy, macro-F1, weighted-F1, per-class precision/recall/F1 and confusion matrices. Hole metrics include accuracy, precision, recall, F1, AUROC, PR-AUC and confusion matrices. Test data was held out from model selection. Reported means are three-seed means for main models and for the no-line-type ablation, while one-seed auxiliary ablations are explicitly labelled.

Reproducibility

The end-to-end experiment is driven by bash scripts/run_full_experiment.sh, and tests are executed with python3 -m pytest tests. Saved outputs include processed datasets, model checkpoints, metrics, predictions, figures and logs under outputs/. Replace the placeholder below with the public repository link before submission.

Course-guideline compliance checklist

Course requirement	Where addressed
Motivation and problem definition	Covered in Abstract, Introduction and Results.
Relevant literature	CC2Vec, PatchNet, DeepJIT and CodeReviewer discussed and cited.
Data description	OpenTitan mining policy, labels, synthetic holes and splits reported.
PyTorch implementation	All neural models and the proposed architecture are PyTorch-based.
Clear baseline	TF-IDF + logistic regression, MLP and BiGRU baselines included.
Model comparison	Main results compare four models across intent and hole metrics.
Ablations	No line-type, no context, single-task and message-only ablations included.
Deep result analysis	Class-level behaviour, confusion matrices, mutation-type recall and errors analysed.
Reproducibility	Run commands, seeds and output artifacts documented.
Team contribution	Placeholders included for completion before submission.
Code repository link	Placeholder included for GitHub URL before submission.

Data availability

The raw source repository is OpenTitan. Derived data files are local project artifacts produced by the mining pipeline: data/raw/opentitan_rtl_commits.jsonl, data/processed/opentitan_real_examples.jsonl and data/processed/opentitan_with_holes.jsonl. If the report is submitted publicly, ensure the derived dataset and scripts are released only in accordance with repository licences and course policy.

Code availability

Code repository: [insert GitHub URL]. The repository should include data-extraction scripts, preprocessing, model definitions, training scripts, evaluation scripts, exact configs, fixed seeds and instructions for reproducing the reported tables and figures.

Author contributions

Team member 1: [fill in exact contributions: data mining, preprocessing, modelling, experiments, analysis, writing].

Team member 2: [fill in exact contributions: data mining, preprocessing, modelling, experiments, analysis, writing].

Competing interests

The authors declare no competing interests.

References

1. lowRISC. OpenTitan: open source silicon Root of Trust. GitHub repository and project documentation. <https://github.com/lowRISC/opentitan>
2. Hoang, T., Kang, H. J., Lawall, J. & Lo, D. CC2Vec: distributed representations of code changes. Proceedings of the 42nd ACM/IEEE International Conference on Software Engineering, 518-529 (2020).
3. Hoang, T., Lawall, J., Tian, Y., Oentaryo, R. J. & Lo, D. PatchNet: hierarchical deep learning-based stable patch identification for the Linux kernel. IEEE Transactions on Software Engineering (2021).
4. Hoang, T., Dam, H. K., Kamei, Y., Lo, D. & Ubayashi, N. DeepJIT: an end-to-end deep learning framework for just-in-time defect prediction. Proceedings of the 16th IEEE/ACM International Conference on Mining Software Repositories, 34-45 (2019).
5. Li, Z. et al. Automating code review activities by large-scale pre-training. Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (2022).